

Pure Pandrosion vectorized via exact telescopic slopes

Mathematical and experimental study of script 115

Ivan Besevic

April 29, 2026

Abstract

We study the engine `115_pandrosion_vectorized_pure_thales_engine.py`. Unlike engine 113, which uses a Jacobian-type local corrector, 115 builds an exact slope matrix $Q(a, b)$ satisfying

$$F(b) - F(a) = Q(a, b)(b - a)$$

via a monomial telescopic identity. This construction is derivative-free at the corrector level: no Jacobian is built for the local correction. The cost of $Q(a, b)$ is reduced through vectorization: tabulated powers, prefix/suffix factors, and a NumPy/BLAS matrix product. The engine retains the same single geometric flow as 113: Riemann-Mobius, Thales-type homothety, starting-point optimization, then correction. On the available tests, 115 recovers 144/144 roots for $ks(2, 12)$, 196/196 for $ks(2, 14)$, 64/64 for $ks(3, 4)$, 81/81 for $ks(4, 3)$, and a strict root for $ks(6, 16)$. Compared to 114, vectorization speeds up the construction of Q by roughly $6\times$ to $7\times$ on the $2, 12$ and $2, 14$ cases.

1 Motivation

Version 113 showed that a powerful geometric flow can extract every root of certain Kostlan systems without explicitly tracking all homotopy paths. But its local corrector solves

$$J_G(y)\Delta = -G(y).$$

Version 115 replaces this step with an exact Pandrosion slope. The goal is to keep the global efficiency of the Thales-Riemann flow while making the local correction faithful to the geometric identity

$$G(b) - G(a) = Q_G(a, b)(b - a).$$

2 Polynomial systems and change of coordinates

Let

$$F = (F_1, \dots, F_n) : \mathbb{C}^n \rightarrow \mathbb{C}^n$$

be a dense polynomial system. The engine uses a linear map

$$z = Ay$$

and solves

$$G(y) = F(Ay) = 0.$$

As in 113, the initial point is generated by a single flow:

$$y_j = \Lambda p_j \frac{\cos(\theta_j)(u_j/p_j) + \sin(\theta_j)}{-\sin(\theta_j)(u_j/p_j) + \cos(\theta_j)}. \quad (1)$$

This map blends homothety, projective inversion, and Riemann rotation in a single formula.

3 Pandrosion slope matrix

Definition 1. Let $a, b \in \mathbb{C}^n$. A matrix $Q_F(a, b) \in \mathbb{C}^{n \times n}$ is a Pandrosion slope for F between a and b if

$$F(b) - F(a) = Q_F(a, b)(b - a). \quad (2)$$

For the transformed system $G(y) = F(Ay)$, we set

$$Q_G(a_y, b_y) = Q_F(Aa_y, Ab_y)A. \quad (3)$$

Then

$$G(b_y) - G(a_y) = Q_G(a_y, b_y)(b_y - a_y).$$

4 Monomial telescopic identity

Consider a monomial

$$m(z) = z^\alpha = \prod_{k=1}^n z_k^{\alpha_k}.$$

For each coordinate j , define

$$S_{j, \alpha_j}(a_j, b_j) = \begin{cases} \sum_{r=0}^{\alpha_j-1} b_j^{\alpha_j-1-r} a_j^r, & \alpha_j > 0, \\ 0, & \alpha_j = 0. \end{cases} \quad (4)$$

Lemma 1 (telescopic slope of a monomial). *We have*

$$m(b) - m(a) = \sum_{j=1}^n \left(\prod_{k < j} b_k^{\alpha_k} \right) S_{j, \alpha_j}(a_j, b_j) \left(\prod_{k > j} a_k^{\alpha_k} \right) (b_j - a_j). \quad (5)$$

Proof. We write the total difference by successively replacing the coordinates of a by those of b :

$$\prod_k b_k^{\alpha_k} - \prod_k a_k^{\alpha_k} = \sum_j \left(\prod_{k < j} b_k^{\alpha_k} \right) (b_j^{\alpha_j} - a_j^{\alpha_j}) \left(\prod_{k > j} a_k^{\alpha_k} \right).$$

Now

$$b_j^q - a_j^q = (b_j - a_j) \sum_{r=0}^{q-1} b_j^{q-1-r} a_j^r.$$

Substituting this identity yields (5). □

For a polynomial

$$F_i(z) = \sum_{\alpha} c_{i, \alpha} z^\alpha,$$

column j of $Q_F(a, b)$ is obtained by summing the contributions (5) weighted by $c_{i, \alpha}$. This construction is exact, up to floating-point error, and requires no derivatives.

5 Pure Pandrosion corrector

At the current iterate a , the engine builds a second point b via a non-infinitesimal geometric formula. It then computes $Q_G(a, b)$ and solves

$$Q_G(a, b)\Delta = -G(a), \quad a_+ = a + \lambda\Delta. \quad (6)$$

The parameter λ is chosen by a proportional search that requires a decrease of the residual. There is no Newton fallback, no least squares, no bivariate specialization, and no backup homotopy.

Remark 1. *The engine still uses `np.linalg.solve`, but only to solve the linear system (6). This is not a hidden root solver.*

6 Vectorization of $Q(a, b)$

Version 114 built $Q(a, b)$ via Python loops. Engine 115 vectorizes the computation.

Let $E \in \mathbb{N}^{M \times n}$ be the matrix of exponents of the M monomials, and let $C \in \mathbb{C}^{n \times M}$ be the matrix of coefficients. For points a, b , we tabulate

$$a_j^m, quad b_j^m, \quad m = 0, \dots, d.$$

Then we build, for all monomials:

$$P_j^<(b) = \prod_{k < j} b_k^{E \cdot k}, \quad (7)$$

$$P_j^>(a) = \prod_{k > j} a_k^{E \cdot k}, \quad (8)$$

$$T_j = P_j^<(b) P_j^>(a) S_{j, E \cdot j}(a_j, b_j). \quad (9)$$

The slope matrix is then obtained by a dense product:

$$Q_F(a, b) = CT. \quad (10)$$

The table $S_{j, m}$ is computed via the recurrence

$$S_{j, 0} = 0, \quad S_{j, m} = b_j^{m-1} + a_j S_{j, m-1}. \quad (11)$$

This recurrence is equivalent to (4).

7 Algorithm

Input: F , a trial budget P , a target count N .

Output: distinct numerical roots.

1. Generate the dense system and the matrices E, C .
2. For each trial, produce u, Λ, θ, p , then y via (1).
3. Improve the starting point via startopt.
4. Repeat:
 - (a) choose a geometric point b associated with a ;
 - (b) build $Q_G(a, b)$ via (3) and (10);
 - (c) solve $Q_G(a, b)\Delta = -G(a)$;
 - (d) apply a proportional search on $a + \lambda\Delta$.
5. Convert $z = Ay$, validate $\|F(z)\|$, then deduplicate.

8 Experimental results

Case	Bezout	Roots requested	Unique roots	Trials	Failures	Time (s)
$ks(2, 12)$	144	144	144	272	124	5.345
$ks(2, 14)$	196	196	196	390	192	5.967
$ks(3, 4)$	64	64	64	125	20	1.586
$ks(4, 3)$	81	81	81	220	57	3.464
$ks(6, 16)$	16,777,216	1	1	1	0	2.639

Table 1: Main results of engine 115.

Case	Q calls	Q time (s)	Max residual	Mean residual
$ks(2, 12)$	6,287	1.024	$9.77 \cdot 10^{-9}$	$3.79 \cdot 10^{-9}$
$ks(2, 14)$	8,624	1.154	$9.96 \cdot 10^{-9}$	$3.99 \cdot 10^{-9}$
$ks(3, 4)$	2,046	0.263	$9.91 \cdot 10^{-9}$	$4.21 \cdot 10^{-9}$
$ks(4, 3)$	3,950	0.655	$9.86 \cdot 10^{-9}$	$4.17 \cdot 10^{-9}$
$ks(6, 16)$	16	1.670	$6.15 \cdot 10^{-13}$	$6.15 \cdot 10^{-13}$

Table 2: Slope statistics and residuals for 115.

9 Comparison with 114

Version 114 used the same telescopic slope idea, but with a less vectorized construction. The available measurements give:

Case	Time 114 (s)	Time 115 (s)	Total speedup	Q speedup
$ks(2, 12)$	11.468	5.230	$2.19\times$	$6.02\times$
$ks(2, 14)$	17.457	6.735	$2.59\times$	$7.43\times$

Table 3: Effect of vectorization compared with 114.

10 Discussion

Vectorization does not change the mathematical content of the slope. It merely shifts the monomial summation toward matrix-matrix product operations. Thus 115 retains the pure Pandrosion character of 114 while substantially reducing the cost of computing Q . The cost remains higher than a classical Jacobian corrector at small degrees, since $Q(a, b)$ encodes a global slope between two points and not just an infinitesimal linearization.

11 Reproducibility

Main commands:

```
python -S 115_pandrosion_vectorized_pure_thales_engine.py \
  --cases 2,12 --equation-normalize --count 144 --pool 5000 \
  --epochs 30 --accept 1e-8 --cluster-sep 1e-8 \
```

```

--startopt-candidates 12 --probe-scale 0.03 --line-search 14

python -S 115_pandrosion_vectorized_pure_thales_engine.py \
  --cases 2,14 --equation-normalize --count 196 --pool 7000 \
  --epochs 30 --accept 1e-8 --cluster-sep 1e-8 \
  --startopt-candidates 12 --probe-scale 0.03 --line-search 14

```

12 Conclusion

Engine 115 provides an autonomous, multivariate, derivative-free (at the local-corrector level) version of Pandrosion. Its mathematical contribution is the vectorized implementation of the exact slope $Q(a, b)$ arising from the monomial telescopic identity. The results show that one preserves the root coverage obtained by the pure engine 114 while substantially accelerating the computation. Engine 115 thus constitutes a base more faithful to Pandrosion geometry than a Newton corrector, while remaining fast enough for dense multivariate tests.